

Lab 6

My solution

main.cpp

```
#include "menuFuncs.hpp"

int main()
{
    int menuChoice = 0;
    do
    {
        displayMenu(menuChoice);
        processMenuChoice(menuChoice);
    } while (menuChoice != 7);

    return 0;
}
```

menuFuncs.hpp

```
#pragma once

int getData();
void displayMenu(int& menuChoice);
void processMenuChoice(int menuChoice);
void isOddEven(int num);
void isPosNeg(int num);
void findNumDigits(int num);
void findDigitAtPosition(int pos);
void displayAdditionTable();
void displayMultiplicationTable();
```

menuFuncs.cpp

```
#include "menuFuncs.hpp"
#include <iostream>
#include <cmath>
#include <iomanip>
using std::cout;
```

```
using std::cin;
using std::setw;

int getData()
{
    int num = 0;

    while (true)
    {
        cin >> num;
        if (num < -1000000 || num > 1000000)
        {
            cout << "Error; please enter a number between -1,000,000 and 1,000,000\
        }
        else {
            break;
        }
    }
    return num;
}

void displayMenu(int& menuChoice)
{
    cout << "1) Is a number odd or even?\n"
        << "2) Is a number positive or negative?\n"
        << "3) How many digits are in a number?\n"
        << "4) What digit is at a specific position?\n"
        << "5) Display the addition table.\n"
        << "6) Display the multiplication table.\n"
        << "7) Exit\n\n";

    cout << "Enter a menu choice: ";
    cin >> menuChoice;
}

void processMenuChoice(int menuChoice)
{
    int tmp = 0;
    switch (menuChoice)
    {
        case 1:
            cout << "Enter a number\n";
            cin >> tmp;
            isOddEven(tmp);
            break;
        case 2:
            cout << "Enter a number\n";
            cin >> tmp;
            isPosNeg(tmp);
```

```
        break;
    case 3:
        cout << "Enter a number\n";
        cin >> tmp;
        findNumDigits(tmp);
        break;
    case 4:
        cout << "Enter a number\n";
        cin >> tmp;
        findDigitAtPosition(tmp);
        break;
    case 5:
        displayAdditionTable();
        break;
    case 6:
        displayMultiplicationTable();
        break;
    case 7:
        cout << "\nThanks for playing!\n\n";
        break;
    default:
        cout << "\nInvalid menu choice\n";
        break;
    }
}

void isOddEven(int num)
{
    if (num % 2 == 0)
    {
        cout << "Number is even\n";
    }
    else {
        cout << "Number is odd\n";
    }
}

void isPosNeg(int num)
{
    if (num < 0)
    {
        cout << "Number is negative\n";
    }
    else if (num > 0)
    {
        cout << "Number is positive\n";
    }
}
```

```
    }
    else {
        cout << "Number is 0\n";
    }
}

void findNumDigits(int num)
{
    int temp = num;
    unsigned short digits = 0;

    if (temp == 0 )
        digits = 1;
    else
    {
        for ( ; temp != 0; digits++ )
            temp /= 10;
    }
    cout << "\nThere are " << digits << " digits.\n\n";
}

void findDigitAtPosition(int num)
{
    int temp = abs(num);
    unsigned short digit_position = 0, digits = 0;

    cout << "Enter the digit position: ";
    cin >> digit_position;

    for ( ; temp != 0 && digits < digit_position - 1; digits++ )
        temp /= 10;

    cout << "\nDigit at position " << digit_position << ": " << temp % 10 << "\n\n"
}

void displayAdditionTable()
{
    cout << '\n' << setw(4) << ' ' ;
    for ( int col = 1; col <= 12; col++ )
        cout << setw(4) << col;

    cout << '\n';

    for ( int row = 1; row <= 12; row++ )
    {
        cout << setw(4) << row;

        for ( int col = 1; col <= 12; col++ )
```

```
        cout << setw(4) << row + col;

        cout << '\n';
    }
    cout << '\n';
}

void displayMultiplicationTable()
{
    cout << '\n' << setw(4) << ' ';
    for ( int col = 1; col <= 12; col++ )
        cout << setw(4) << col;

    cout << '\n';

    for ( int row = 1; row <= 12; row++ )
    {
        cout << setw(4) << row;

        for ( int col = 1; col <= 12; col++ )
            cout << setw(4) << row * col;

        cout << '\n';
    }
    cout << '\n';
}
```

AI Feedback

Your program is organized into functions as required, and each function is clean, readable, and logically separated. The menu system works, and your addition/multiplication tables are formatted well. There are, however, several issues and opportunities for improvement:

1. Some functions don't follow the assignment specification

- `findNumDigits` and `findDigitAtPosition` **should return values**, not print them directly. The assignment explicitly states:

“Both functions return a value, which is then displayed in the `ProcessMenuChoice` function.”

2. `findDigitAtPosition` has the wrong parameters

The spec says:

Parameters: (number, position)

Your function currently takes only `num` and asks the user for the position inside the function.

3. `processMenuChoice()` should use `getData()`

Right now you do:

```
cin >> tmp;
```

instead of calling your own validated input function.

4. `menuFuncs.hpp` contains incorrect signatures

`findNumDigits` and `findDigitAtPosition` are declared as:

```
void findNumDigits(int num);  
void findDigitAtPosition(int pos);
```

but based on the assignment:

- both should return an integer
- `findDigitAtPosition` needs two params: (int num, int pos)

5. Input validation could be improved

`displayMenu` should verify the menu choice is within 1–7.

6. Minor style improvements

- Use consistent indentation.
- Avoid repeating `cout << "Enter a number\n";` multiple times.
- Consider using `switch` fallthrough guards + early returns for clarity.

Rewritten Code (After Applying Feedback)

This is the “improved” version you’ll include after the feedback section.

main.cpp

```
#include "menuFuncs.hpp"

int main() {
    int menuChoice = 0;

    do {
        displayMenu(menuChoice);
        processMenuChoice(menuChoice);
    } while (menuChoice != 7);

    return 0;
}
```

menuFuncs.hpp

```
#pragma once

int getData();
void displayMenu(int& menuChoice);
void processMenuChoice(int menuChoice);

void isOddEven(int num);
void isPosNeg(int num);

int findNumDigits(int num);
int findDigitAtPosition(int num, int pos);

void displayAdditionTable();
void displayMultiplicationTable();
```

menuFuncs.cpp

```
#include "menuFuncs.hpp"
#include <iostream>
#include <iomanip>
#include <cmath>
```

```
using std::cin;
using std::cout;
using std::setw;

int getData() {
    int num;

    while (true) {
        cout << "Enter a number (-1,000,000 to 1,000,000): ";
        cin >> num;

        if (!cin) {
            cin.clear();
            cin.ignore(9999999, '\n');
            cout << "Invalid input. Try again.\n";
            continue;
        }

        if (num < -1000000 || num > 1000000) {
            cout << "Error: Out of range.\n";
        } else {
            break;
        }
    }
    return num;
}

void displayMenu(int& menuChoice) {
    cout << "\n1) Is a number odd or even?\n"
        << "2) Is a number positive or negative?\n"
        << "3) How many digits does a number have?\n"
        << "4) What digit is at a specific position?\n"
        << "5) Display the addition table.\n"
        << "6) Display the multiplication table.\n"
        << "7) Exit\n\n"
        << "Enter a menu choice: ";

    cin >> menuChoice;

    while (menuChoice < 1 || menuChoice > 7 || !cin) {
        cin.clear();
        cin.ignore(9999999, '\n');
        cout << "Invalid choice. Enter 1-7: ";
        cin >> menuChoice;
    }
}
```

```
void processMenuChoice(int menuChoice) {
    int num, pos;

    switch (menuChoice) {
        case 1:
            num = getData();
            isOddEven(num);
            break;

        case 2:
            num = getData();
            isPosNeg(num);
            break;

        case 3:
            num = getData();
            cout << "Digits: " << findNumDigits(num) << "\n";
            break;

        case 4:
            num = getData();
            cout << "Enter digit position: ";
            cin >> pos;
            cout << "Digit: " << findDigitAtPosition(num, pos) << "\n";
            break;

        case 5:
            displayAdditionTable();
            break;

        case 6:
            displayMultiplicationTable();
            break;

        case 7:
            cout << "Goodbye!\n";
            break;
    }
}

void isOddEven(int num) {
    if (num == 0) cout << "Zero is even.\n";
    else if (num % 2 == 0) cout << "Number is even.\n";
    else cout << "Number is odd.\n";
}

void isPosNeg(int num) {
    if (num > 0) cout << "Number is positive.\n";
}
```

```
    else if (num < 0) cout << "Number is negative.\n";
    else cout << "Number is zero.\n";
}

int findNumDigits(int num) {
    num = std::abs(num);
    if (num == 0) return 1;

    int count = 0;
    while (num > 0) {
        num /= 10;
        count++;
    }
    return count;
}

int findDigitAtPosition(int num, int pos) {
    num = std::abs(num);

    for (int i = 1; i < pos; i++)
        num /= 10;

    return num % 10;
}

void displayAdditionTable() {
    cout << '\n' << setw(4) << ' ';
    for (int c = 1; c <= 12; c++) cout << setw(4) << c;
    cout << '\n';

    for (int r = 1; r <= 12; r++) {
        cout << setw(4) << r;
        for (int c = 1; c <= 12; c++) cout << setw(4) << (r + c);
        cout << '\n';
    }
}

void displayMultiplicationTable() {
    cout << '\n' << setw(4) << ' ';
    for (int c = 1; c <= 12; c++) cout << setw(4) << c;
    cout << '\n';

    for (int r = 1; r <= 12; r++) {
        cout << setw(4) << r;
        for (int c = 1; c <= 12; c++) cout << setw(4) << (r * c);
        cout << '\n';
    }
}
```

